

Theme Report 2

Reason

COMPENG 2DX3

Claresta Cheung

400582489

March 13th, 2025

Table of Contents

Table of Contents.....	2
Theme.....	2
Background.....	2
Theme Exemplars.....	3
Lab 4 - PWM and Stepper Motor.....	3
Lab 5 - Keypad Interfacing.....	4
Debugging exemplar.....	5
Synthesis.....	7
Reflection.....	8
References.....	9

Theme

The theme of this report is Reason - in which it includes lab exercises 4-6. Reasoning is one of three fundamental properties of an intelligent micro system - centered around how such a system can make decisions based upon the world around it. This theme is demonstrated through real-world technologies, such as game-playing AI, which uses logical reasoning to come up with possible moves and outcomes to select the best strategy in winning the game.

Background

Reasoning is important for intelligent microsystems to process real-world data through logic reasoning, such as: boolean logic and machine learning. Without reliable reasoning, a system cannot accurately provide an appropriate response to changing conditions in the environment. Modern systems require continuous reasoning of the world to improve system performance, efficiency, and user experience.

Lab 4 exercises required simple logic of code to adjust LED intensity and rotation of the stepper motor. Lab 5 exercises required keypad logic for the microcontroller to identify user input. Lab 6 exercises include a combination of all logic and hardware based knowledge acquired from previous labs. These labs demonstrated how a microcontroller performs reasoning after collecting data of its surroundings.

Theme Exemplars

Lab 4 - PWM and Stepper Motor

The first example selected from my lab work in relation to the theme - Reason - is the Bonus Milestone from Lab 4. This exercise required the modification of the RGB color of the LED based upon any percent duty-cycle input for each of the Red, Green, and Blue inputs. Based on the new percentages of each, provided by the TA, the microcontroller reasoned through what the user inputs. According to these inputs, a corresponding color hue is produced. This is demonstrated in *Figure 1*, in which the color purple is produced based on the percent values inputted into the code for the microcontroller to reason. For example, when controlling this RGB LED intensity with PWM:

Red duty cycle = 60%

Blue duty cycle = 60%

Green duty cycle = 0%

This produces a purple color since both red and blue LEDs are on, while green remains off.

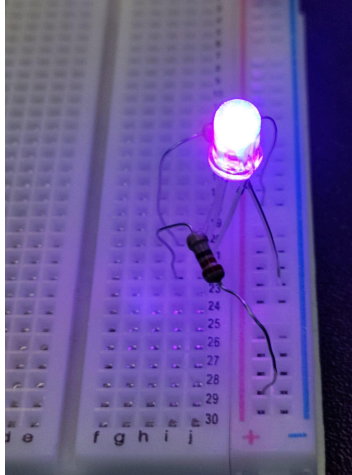


Figure 1: Purple color on RGB

Lab 5 - Keypad Interfacing

The second example selected from my lab work in relation to the theme - Reason - is Milestone 1 Exercise from Lab 5. This exercise required the microprocessor to determine which key has been pressed by the user, then store the corresponding 8-bit code of the scan in Keil memory. This value is seen through the watch table of the variable `scan_code` (*Figure 2*).

The microcontroller processes and reasons which button is pressed through the use of the 'if-statement' in *Figure 3*. The logic of this code is that when a 0 is detected via a specific row and column connection, that corresponding row shows a button 'on' for that. Once this occurs, the variable `scan_code` is updated and shown in real-time in the watch table.

```

1  #include <stdint.h>
2  #include "tm4cl294ncpdt.h"
3  #include "Systick.h"
4  #include "PLL.h"
5
6  // GLOBAL VARIABLE for Keil Debugger (Watch 1)
7  // This will store the 8-bit code (PM[3:0] PE[3:0]) requested in Milestone 7.1
8  volatile uint8_t scan_code = 0xFF;
9
10 void PortE_Init(void){
11     SYSTCL_RCGCGPIO_R |= SYSTCL_RCGCGPIO_R4;           // Activate the clock for Port E
12     while((SYSTCL_PRGPIO_R & SYSTCL_PRGPIO_R4) == 0){}; // Allow time for clock to stabilize
13
14     GPIO_PORTE_DIR_R = 0b00001111;                     // Enable PE0-PE3 as OUTPUTS (Rows)
15     GPIO_PORTE_DEN_R = 0b00001111;                     // Enable PE0-PE3 as digital pins
16 }
17
18 void PortM_Init(void){
19     SYSTCL_RCGCGPIO_R |= SYSTCL_RCGCGPIO_R11;          // Activate the clock for Port M
20     while((SYSTCL_PRGPIO_R & SYSTCL_PRGPIO_R11) == 0){}; // Allow time for clock to stabilize
21
22     GPIO_PORTM_DIR_R = 0b00000000;                     // FIXED: Enable PM0-PM3 as INPUTS (Columns)
23     GPIO_PORTM_DEN_R = 0b00001111;                     // Enable PM0-PM3 as digital pins
24     GPIO_PORTM_PUR_R = 0b00001111;                     // FIXED: Enable Pull-Up resistors on PM0-PM3
25 }
26
27 int main(void){
28     // Initialize standard clock and ports
29     PLL_Init();
30     PortE_Init();
31     PortM_Init();
32 }

```

Figure 2: Variable `scan_code`

```

// If read_col is NOT 1111 (0x0F), a button is being pressed in this row!
if(read_col != 0x0F) {

    // MILESTONE 7.1: Combine Port M (Columns) and Port E (Rows) into an 8-bit code
    // Shift Port M left by 4 bits, then OR it with Port E.
    scan_code = (read_col << 4) | current_row;

    // Wait here until the user RELEASES the button so we don't spam the debugger
    while((GPIO_PORTM_DATA_R & 0x0F) != 0x0F);

}

}

}

```

Figure 3: If statement

Debugging exemplar

The debug exemplar from my lab work in relation to the theme - Reason - is Lab 6 Deliverable 1.

In this Deliverable, both hardware and software debugging was done. As this Lab required more

functions implemented in comparison to the previous 5 labs, it would sometimes be frustrating when something was wired incorrectly or a line of code would throw off the entire implementation. If a part of the code seemed logically sound but didn't work as expected in terms of the requirements, much patience and sometimes trial and error was involved.

A specific debug example that is different from the previous theme report - Observe - would be a hardware debugging problem. After implementing the code and wiring, it was time to test out the configuration through flashing to the microcontroller. However, the motor was vibrating instead of rotating normally.

The reason why this happened was because pins PH2 and PH3 were switched. Although the code implementation was correct, the inaccurate pin wiring caused the motor to spin incorrectly. The reasoning of the microcontroller was correct, but this doesn't always indicate perfect results when testing it out experimentally. This was demonstrated through the hardware issue, which was then resolved by switching pins PH2 and PH3 (*Figure 4*) to allow for proper mechanical resolution while the microcontroller reasoned.

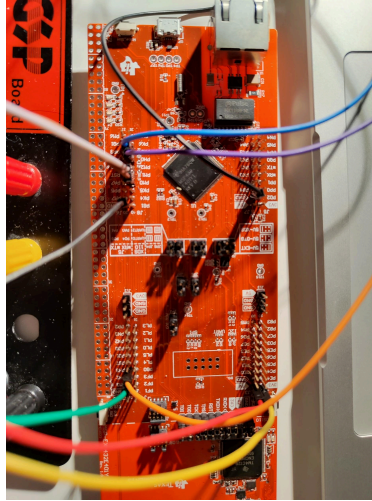


Figure 4: Correct Wiring on Microcontroller

Synthesis

Across Labs 4 to 6, all three selected theme examples collectively illustrate the theme of Reasoning in the context of intelligent systems. The microcontroller consistently performs the same fundamental task - going through the code (if else statements) to make the most accurate assessment of the next direction it should take. For each task, the microcontroller goes through a structured decision-making process based on the logic/rules encoded in the program.

Although each exercise involved different hardware components, the underlying mechanism remained the same: evaluation of inputs using programmed logical rules and microcontroller reasoning to determine the appropriate response. Reasoning in intelligent microsystems isn't just

only if-else statements, but it requires structured evaluation of inputs, system states, and hardware interactions.

As a whole, these examples enable intelligent systems to gain a more logical approach after data and environmental observation. This allows for the system to move on to the next step - Act - in order to come to a right conclusion of their next execution of program behavior.

Reflection

These examples contributed to my understanding of this theme of Reasoning as I went through the exercises in Labs 4, 5, and 6. The first two labs mentioned provided me with a foundational understanding of how a microprocessor reasons the digital input observed. By putting myself in the frame of reference of the microprocessor, I was able to get through the logic of how the program 'thinks' when an input is fed in, and an output is expected.

Lab 4 demonstrated simple logical reasoning of how different colors can be merged through varying percentages sent as input to the microcontroller to come to a conclusion (output). This became the critical stepping stone in understanding the straightforward way a machine approaches a problem when reasoning.

Lab 5 was a little trickier than Lab 4, given that a greater number of inputs had to be monitored and updated in real time. The microcontroller had to reason through binary logic to come to a conclusion of which button was pressed by the user.

Lab 6 becomes the main factor in solidifying this theme of Reasoning. A lot of reasoning was done in this Lab, which was built off of previous labs. Some of the reasoning done was a little trickier as well, since some functions (button presses) influenced one another. For example, when PJ1 is pressed, the amount the stepper motor has to rotate must reset back to 0 degrees and turn 360 degrees. Originally, I mistook the instructions as: complete the rest of the 360 degrees without resetting back to 0 degrees as the beginning.

References

- [1] Y. M. Haddara, *2025–2026 COMPENG 2DX3 Laboratory Manual*, Jan. 2024. [Online].
Available: <https://avenue.cllmcmaster.ca/d2l/le/content/731113/viewContent/5518323/View>.
Accessed: Mar. 13, 2026.